

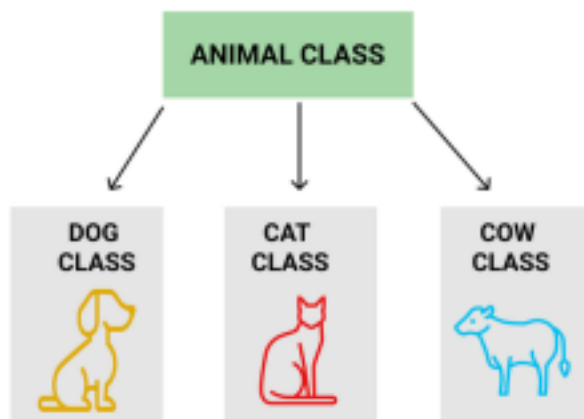
UNIT III INHERITANCE AND POLYMORPHISM

Types of Inheritance in java (Single, Multilevel, Heirarchical), Method overriding and runtime polymorphism, super keyword and constructor chaining, Abstract classes and methods, Interfaces and multiple inheritance through interfaces, Introduction to Packages and access control, Applications: Employee Management, vehicle rental systems.

3.1 INHERITANCE IN JAVA

Java Inheritance is a fundamental concept in OOP(Object-Oriented Programming). It is the mechanism in Java by which one class is allowed to inherit the features (fields and methods) of another class. In Java, Inheritance means creating new classes based on existing ones. A class that inherits from another class can reuse the methods and fields of that class.

Example: In the following example, Animal is the base class and Dog, Cat and Cow are derived classes that extend the Animal class.



Implementation:

```
// Parent class
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

// Child class
class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}
```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

```
}  
}  
// Child class  
class Cat extends Animal {  
    void sound() {  
        System.out.println("Cat meows");  
    }  
}  
// Child class  
class Cow extends Animal {  
    void sound() {  
        System.out.println("Cow moos");  
    }  
}  
// Main class  
public class mainclass {  
    public static void main(String[] args) {  
        Animal a;  
        a = new Dog();  
        a.sound();  
        a = new Cat();  
        a.sound();  
        a = new Cow();  
        a.sound();  
    }  
}
```

Output

Dog barks

Cat meows

Cow moos

Explanation:

- Animal is the base class.
- Dog, Cat and Cow are derived classes that extend Animal class and provide specific

implementations of the sound() method.

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY

UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

- The Mainclass is the driver class that creates objects and demonstrates runtime polymorphism using method overriding.

***Note:** In practice, inheritance and polymorphism are used together in Java to achieve fast performance and readability of code.*

Syntax

```
class ChildClass extends ParentClass {  
    // Additional fields and methods  
}
```

***Note:** In Java, inheritance is implemented using the extends keyword. The class that inherits is called the subclass (child class) and the class being inherited from is called the superclass (parent class).*

Why Use Inheritance in Java?

- **Code Reusability:** The code written in the Superclass is common to all subclasses. Child classes can directly use the parent class code.
- **Method Overriding:** Method Overriding is achievable only through Inheritance. It is one of the ways by which Java achieves Run Time Polymorphism.
- **Abstraction:** The concept of abstraction where we do not have to provide all details, is achieved through inheritance. Abstraction only shows the functionality to the user. **Key**

Terminologies Used in Java Inheritance

- **Class:** Class is a set of objects that share common characteristics/ behavior and common properties/ attributes. Class is not a real-world entity. It is just a template or blueprint or prototype from which objects are created.
- **Super Class/Parent Class:** The class whose features are inherited is known as a superclass(or a base class or a parent class).
- **Sub Class/Child Class:** The class that inherits the other class is known as a subclass(or a derived class, extended class or child class). The subclass can add its own fields and methods in addition to the superclass fields and methods.

- **Extends Keyword:** This keyword is used to inherit properties from a superclass.

How Inheritance Works in Java?

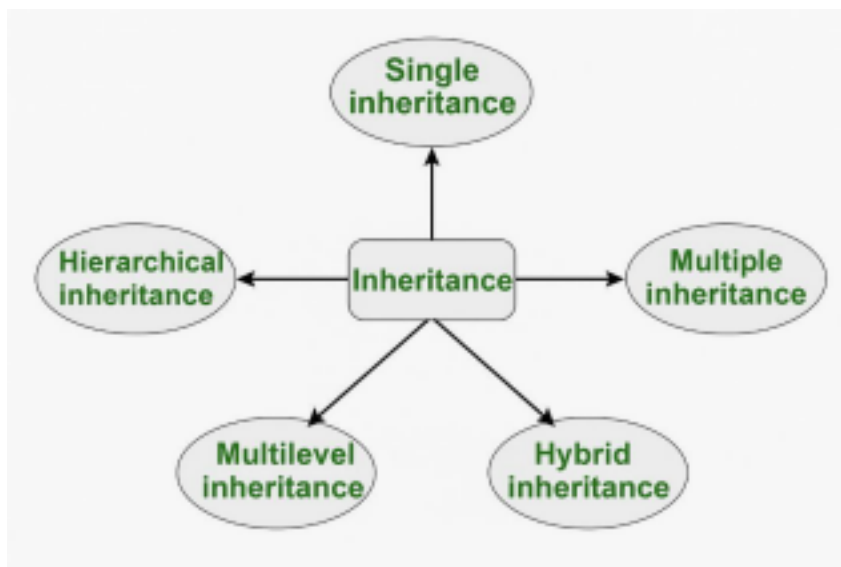
ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY

UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

The **extends keyword** is used for inheritance in Java. It enables the subclass to inherit the fields and methods of the superclass. When a class extends another class, it means it inherits all the non-primitive members (fields and methods) of the parent class and the subclass can also override or add new functionality to them.

***Note:** The extends keyword establishes an "is-a" relationship between the child class and the parent class. This allows a child class to have all the behavior of the parent class.*

Types of Inheritance in Java



3.1.1 Types of Inheritance in Java

Below are the different types of inheritance which are supported by Java. •

Single Inheritance

- Multilevel Inheritance
- Hierarchical Inheritance
- Multiple Inheritance
- Hybrid Inheritance

1. Single Inheritance

In single inheritance, a sub-class is derived from only one super class. It inherits the properties

and behavior of a single-parent class. Sometimes, it is also known as simple inheritance.

Example:

```
//Super class
class Vehicle {
    Vehicle() {
```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY

UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

```
        System.out.println("This is a Vehicle");
    }
}
// Subclass
class Car extends Vehicle {
    Car() {
        System.out.println("This Vehicle is Car");
    }
}
public class Test {
    public static void main(String[] args) {
        // Creating object of subclass invokes base class constructor
        Car obj = new Car();
    }
}
```

Output

This is a Vehicle

This Vehicle is Car

2. Multilevel Inheritance

In Multilevel Inheritance, a derived class will be inheriting a base class and as well as the derived class also acts as the base class for other classes.

Example:

```
class Vehicle {
    Vehicle() {
        System.out.println("This is a Vehicle");
    }
}
class FourWheeler extends Vehicle {
    FourWheeler() {
```

```

System.out.println("4 Wheeler Vehicles");
}
}
class Car extends FourWheeler {
    Car() {
        System.out.println("This 4 Wheeler Vehicle is a Car");
    }
}

```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY

UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

```

}
public class mclass {
    public static void main(String[] args) {
        Car obj = new Car(); // Triggers all constructors in order
    }
}

```

Output

This is a Vehicle

4 Wheeler Vehicles

This 4 Wheeler Vehicle is a Car

3. Hierarchical Inheritance

In hierarchical inheritance, more than one subclass is inherited from a single base class. i.e. more than one derived class is created from a single base class. For example, cars and buses both are vehicle

Example:

```

class Vehicle {
    Vehicle() {
        System.out.println("This is a Vehicle");
    }
}
class Car extends Vehicle {
    Car() {
        System.out.println("This Vehicle is Car");
    }
}
class Bus extends Vehicle {

```

```

Bus() {
    System.out.println("This Vehicle is Bus");
}
}
public class Test {
    public static void main(String[] args) {
        Car obj1 = new Car();
        Bus obj2 = new Bus();
    }
}

```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

```

}

```

Output

This is a Vehicle

This Vehicle is Car

This is a Vehicle

This Vehicle is Bus

4. Multiple Inheritance (Through Interfaces)

In Multiple inheritances, one class can have more than one superclass and inherit features from all parent classes.

***Note:** that Java does not support multiple inheritances with classes. In Java, we can achieve multiple inheritances only through Interfaces.*

Example:

```

interface LandVehicle {
    default void landInfo() {
        System.out.println("This is a LandVehicle");
    }
}
interface WaterVehicle {
    default void waterInfo() {
        System.out.println("This is a WaterVehicle");
    }
}

```

```
// Subclass implementing both interfaces
class AmphibiousVehicle implements LandVehicle, WaterVehicle {
    AmphibiousVehicle() {
        System.out.println("This is an AmphibiousVehicle");
    }
}

public class Test {
    public static void main(String[] args) {
        AmphibiousVehicle obj = new AmphibiousVehicle();
        obj.waterInfo();
        obj.landInfo();
    }
}
```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY

UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

```
}
}
```

Output

This is an AmphibiousVehicle

This is a WaterVehicle

This is a LandVehicle

5. Hybrid Inheritance

It is a mix of two or more of the above types of inheritance. In Java, we can achieve hybrid inheritance only through Interfaces if we want to involve multiple inheritance to implement Hybrid inheritance.

Java IS-A type of Relationship

IS-A represents an inheritance relationship in Java, meaning this object is a type of that object.

```
public class SolarSystem {
}

public class Earth extends SolarSystem {
}

public class Mars extends SolarSystem {
}

public class Moon extends Earth {
```

```
}
```

Now, based on the above example, in Object-Oriented terms, the following are true: •

SolarSystem is the superclass of Earth class.

• SolarSystem is the superclass of Mars class.

• Earth and Mars are subclasses of SolarSystem class.

• Moon is the subclass of both Earth and SolarSystem classes.

```
class SolarSystem {  
}  
class Earth extends SolarSystem {  
}
```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY

UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

```
class Mars extends SolarSystem {  
}  
public class Moon extends Earth {  
    public static void main(String args[])  
    {  
        SolarSystem s = new SolarSystem();  
        Earth e = new Earth();  
        Mars m = new Mars();  
        System.out.println(s instanceof SolarSystem);  
        System.out.println(e instanceof Earth);  
        System.out.println(m instanceof SolarSystem);  
    }  
}
```

Example : Write a Java program to a class Person stores basic details like name and age, a class Employee (derived from Person) stores employee ID and department, and a class Manager (derived from Employee) stores the number of team members.(Inheritance)

// Person class - base class

```
class Person {  
    private String name;  
    private int age;
```

```

public Person(String name, int age) {
this.name = name;
this.age = age;
}
public String getName() {
return name;
}
public int getAge() {
return age;
}

```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

```

public void displayPersonDetails() {
System.out.println("Name: " + name);
System.out.println("Age: " + age);
}
}
// Employee class - derived from Person
class Employee extends Person {
private String employeeId;
private String department;
public Employee(String name, int age, String employeeId, String department) {
super(name, age); // Call the constructor of the superclass (Person)
this.employeeId = employeeId;
this.department = department;
}
public String getEmployeeId() {
return employeeId;
}
}

```

```

}

public String getDepartment() {
return department;
}

public void displayEmployeeDetails() {
displayPersonDetails(); // Call method from superclass
System.out.println("Employee ID: " + employeeId);
System.out.println("Department: " + department);
}
}

// Manager class - derived from Employee

```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

```

class Manager extends Employee {
private int numberOfTeamMembers;

public Manager(String name, int age, String employeeId, String department, int
numberOfTeamMembers) {

super(name, age, employeeId, department); // Call the constructor of the superclass (Employee)
this.numberOfTeamMembers = numberOfTeamMembers;
}

public int getNumberOfTeamMembers() {
return numberOfTeamMembers;
}

public void displayManagerDetails() {
displayEmployeeDetails(); // Call method from superclass
System.out.println("Number of Team Members: " + numberOfTeamMembers); }
}

// Main class to demonstrate the inheritance

```

```

public class CompanyHierarchy {

    public static void main(String[] args) {

        // Create a Person object

        Person person1 = new Person("Alice", 30);

        System.out.println("--- Person Details ---");

        person1.displayPersonDetails();

        System.out.println();

        // Create an Employee object

        Employee employee1 = new Employee("Bob", 25, "EMP001", "IT");

        System.out.println("--- Employee Details ---");

        employee1.displayEmployeeDetails();

```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

```

        System.out.println();

        // Create a Manager object

        Manager manager1 = new Manager("Charlie", 40, "MGR001", "HR", 10);

        System.out.println("--- Manager Details ---");

        manager1.displayManagerDetails();

    }}

```

3.2 METHOD OVERRIDING

Overriding in Java occurs when a subclass or child class implements a method that is already defined in the superclass or base class. When a subclass provides its own version of a method that is already defined in its superclass, we call it method overriding. The subclass method must match the parent class method's name, parameters, and return type.

Rules for Overriding:

- Name, parameters, and return type must match the parent method.

Example: In the code below, Dog overrides the move() method from Animal but keeps eat() as it is. When we call move() on a Dog object, it runs the dog-specific version.

// Example of Overriding in Java

```
class Animal {  
    void move()  
    {  
        System.out.println("Animal is moving.");  
    }  
    void eat()  
    {  
        System.out.println("Animal is eating.");  
    }  
}
```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY

UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

```
class Dog extends Animal {  
    {  
        System.out.println("Dog is running.");  
    }  
    void bark()  
    {  
        System.out.println("Dog is barking.");  
    }  
}  
public class mclass {  
    public static void main(String[] args)  
    {  
        Dog d = new Dog();  
        d.move(); // Output: Dog is running.  
        d.eat(); // Output: Animal is eating.
```

```
d.bark(); // Output: Dog is barking.
```

```
}
```

```
}
```

Output

Dog is running.

Animal is eating.

Dog is barking.

Explanation: The Animal class defines base functionalities like move() and eat(). The Dog class inherits from Animal and overrides the move() method to provide a specific behavior Dog is running. Both classes can access their own methods. When creating a Dog object, calling move() executes the overridden method.

Example: This example demonstrates runtime polymorphism in Java, where the show() method is overridden in the Child class, and the method called depends on the object type at runtime.

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY

UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

```
// Java program to demonstrate method overriding in java
```

```
class Parent {
```

```
    void show()
```

```
    { System.out.println("Parent's show()");
```

```
    }
```

```
}
```

```
class Child extends Parent {
```

```
    {
```

```
        System.out.println("Child's show()");
```

```
    }
```

```
}
```

```
class class1 {
```

```
    public static void main(String[] args)
```

```

{
Parent obj1 = new Parent();
obj1.show();
Parent obj2 = new Child();
obj2.show();
}
}

```

Output

Parent's show()

Child's show()

Explanation: In this code, the Child class inherits from the Parent class and overrides the show() method, providing its own implementation. When a Parent reference points to a Child object, the Child's overridden show() method is executed at runtime, showcasing the principle of polymorphism in Java.

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

Example problem

Write a Java program for an employee management system where subclasses override a method to display employee details differently for permanent and temporary employees.(Method overriding)

```

class Employee {
String name;
int employeeId;
double salary;
public Employee(String name, int employeeId, double salary) {
this.name = name;
this.employeeId = employeeId;
this.salary = salary;
}
}

```

```

    }

    public void displayDetails() {
        System.out.println("Employee Name: " + name);
        System.out.println("Employee ID: " + employeeId);
        System.out.println("Base Salary: $" + salary);
    }
}

// PermanentEmployee.java

class PermanentEmployee extends Employee {
    double bonus;

    public PermanentEmployee(String name, int employeeId, double salary, double bonus) {
        super(name, employeeId, salary);
        this.bonus = bonus;
    }

    public void displayDetails() {

```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

```

        super.displayDetails(); // Call superclass method to display common details
        System.out.println("Employee Type: Permanent");
        System.out.println("Bonus: $" + bonus);
        System.out.println("Total Salary: $" + (salary + bonus));
    }
}

// TemporaryEmployee.java

class TemporaryEmployee extends Employee {
    int contractDurationMonths; // in months

    public TemporaryEmployee(String name, int employeeId, double salary, int
contractDurationMonths) {
        super(name, employeeId, salary);

```

```

this.contractDurationMonths = contractDurationMonths;
}

public void displayDetails() {
super.displayDetails(); // Call superclass method to display common details
System.out.println("Employee Type: Temporary");

System.out.println("Contract Duration: " + contractDurationMonths + " months"); }}

// EmployeeManagementSystem.java

public class EmployeeManagementSystem {
public static void main(String[] args) {
Employee permEmployee = new PermanentEmployee("Alice Smith", 101, 50000, 5000);
Employee tempEmployee = new TemporaryEmployee("Bob Johnson", 201, 30000, 6);
System.out.println("--- Permanent Employee Details ---");

permEmployee.displayDetails();

System.out.println("\n--- Temporary Employee Details ---");

tempEmployee.displayDetails(); }}

```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY

UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

3.3 SUPER KEYWORD AND CONSTRUCTOR CHAINING

3.3.1 Super Keyword in Java

The super keyword in Java is a reference variable that is used to refer to the parent class when we are working with objects. You need to know the basics of Inheritance and Polymorphism to understand the Java super keyword.

Characteristics of Super Keyword in Java

The characteristics of the super keyword are listed below:

- **Calling the Parent Class Constructor:** When we create an object of the subclass, its constructor needs to call the constructor of the parent class. This can be done with the help of the super keyword and it calls the constructor of the parent class.

- **Accessing Parent Class Methods:** If the subclass wants to access the methods of the parent class, it can also be done with the help of the super keyword.
- **Accessing Parent Class Fields:** Fields from the parent class can also be accessed using the super keyword in the subclass.
- **First Statement in a Constructor:** When calling a superclass constructor, the super() statement must be the first statement in the constructor of the subclass. This ensures that the parent class is properly initialized before the subclass does anything else.
- **Cannot be used in Static Context:** We cannot use super in a static variable, static method and static block.

Use of super Keyword in Java

Super Keyword are mainly used in the following contexts which are listed

below: **1. Use of super with Variables**

Referring to immediate parent class instance variables: When a subclass and its immediate parent class have instance variables with the same name, super.variableName is used to differentiate and access the parent class's variable. This resolves ambiguity and allows access to the inherited member.

Program:

```
class Vehicle {
    int maxSpeed = 120;
}
```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY

UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

```
class Car extends Vehicle {
    int maxSpeed = 180;
    void display()
    {
        System.out.println("Maximum Speed: "+ super.maxSpeed);
    }
}

class Test {
    public static void main(String[] args)
    {
```

```

Car small = new Car();
small.display();
}
}

```

Output

Maximum Speed: 120

Explanation: In the above example, both the base class and subclass have a member **maxSpeed**. We could access the maxSpeed of the base class in subclass using super keyword.

2. Use of super with Methods

This is used when we want to call the parent class method. So, whenever a parent and child class have the same-named methods then to resolve ambiguity we use the super keyword. Invoking immediate parent class methods: If a subclass overrides a method from its parent class, super.methodName() can be used within the subclass to explicitly call the overridden method of the parent class. This allows extending or augmenting the parent's behavior. Program:

```

class Person {
    void message()
    {
        System.out.println("This is person class\n");
    }
}

```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

```

}
class Student extends Person {
    void message()
    {
        System.out.println("This is student class");
    }
    void display()
    {
        message();
    }
}

```

```

super.message();
}
}
class Test {
public static void main(String args[])
{
Student s = new Student();
s.display();
}
}

```

Output

This is student class

This is person class

Explanation: In the above example, we have seen that if we only call method **message()** then, the current class message() is invoked but with the use of the super keyword, message() of the superclass could also be invoked.

3. Use of super with Constructors. (Constructor Chaining)

The super keyword can also be used to access the parent class constructor. One more important thing is that ‘super’ can call both parametric as well as non-parametric constructors depending on the situation.

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY

UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

The super() call (with or without arguments) is used within a subclass constructor to invoke the constructor of its immediate parent class.

```

class Person {
Person()
{
System.out.println("Person class Constructor");
}
}

```

```
class Student extends Person {  
    Student()  
    {  
        super();  
        System.out.println("Student class Constructor");  
    }  
}  
  
class Test {  
    public static void main(String[] args)  
    {  
        Student s = new Student();  
    }  
}
```

Output

Person class Constructor

Student class Constructor

Explanation: In the above example, we have called the superclass constructor using the keyword "super" via subclass constructor.

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

3.3.2 Constructor Chaining:

Constructor chaining is the process of calling one constructor from another constructor with respect to current object.

One of the main use of constructor chaining is to avoid duplicate codes while having multiple constructor (by means of constructor overloading) and make code more readable.

Constructor chaining can be done in two ways:

- **Within same class:** It can be done using **this()** keyword for constructors in the same class •
- From base class:** by using **super()** keyword to call the constructor from the base class.

Constructor chaining occurs through **inheritance**. A sub-class constructor's task is to call super class's constructor first. This ensures that the creation of sub class's object starts with the initialization of the data members of the superclass. There could be any number of classes in the inheritance chain. Every constructor calls up the chain till the class at the top is reached.

Why do we need constructor chaining?

This process is used when we want to perform multiple tasks in a single constructor rather than creating a code for each task in a single constructor we create a separate constructor for each task and make their chain which makes the program more readable.

Constructor Chaining within the same class using this() keyword:

// Java program to illustrate Constructor Chaining within same class Using this() keyword

```
class Temp
```

```
{
```

```
// default constructor 1
```

```
// default constructor will call another constructor using this keyword from same class
```

```
Temp()
```

```
{
```

```
// calls constructor 2
```

```
this(5);
```

```
System.out.println("The Default constructor");
```

```
}
```

```
// parameterized constructor 2
```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY

UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

```
Temp(int x)
```

```
{
```

```
// calls constructor 3
```

```
this(5, 15);
```

```

System.out.println(x);
}
// parameterized constructor 3
Temp(int x, int y)
{
System.out.println(x * y);
}
public static void main(String args[])
{
// invokes default constructor first
new Temp();
}
}

```

Output:

75

5

The Default constructor

Constructor Chaining to other class using super() keyword :

// Java program to illustrate Constructor Chaining to // other

class using super() keyword

class Base

```

{
String name;

```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY

UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

// constructor 1

Base()

```

{

```

```

this("");
System.out.println("No-argument constructor of" + " base class"); }

// constructor 2
Base(String name)
{
this.name = name;
System.out.println("Calling parameterized constructor" + " of base"); }
}

class Derived extends Base
{
// constructor 3
Derived()
{
System.out.println("No-argument constructor " + "of derived"); }

// parameterized constructor 4
Derived(String name)
{
// invokes base class constructor 2
super(name);
System.out.println("Calling parameterized " + "constructor of derived"); }
}

```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

```

public static void main(String args[])
{
// calls parameterized constructor 4
Derived obj = new Derived("test");
// Calls No-argument constructor
}

```

```
// Derived obj = new Derived();  
  
}  
  
}
```

Output:

Calling parameterized constructor of base

Calling parameterized constructor of derived

3.4 ABSTRACT CLASSES AND METHODS

In Java, abstract classes and methods are fundamental concepts for achieving abstraction in object-oriented programming.

Abstract Class:

An abstract class is a class declared with the abstract keyword. Key characteristics include:

- **Cannot be instantiated:** You cannot create objects directly from an abstract class. It serves as a blueprint for other classes.
- **Can contain abstract methods:** Methods declared without an implementation (body).
- **Can contain concrete methods:** Methods with full implementations that can be inherited by subclasses.
- **Can have constructors:** These are invoked when a subclass is instantiated, allowing for initialization of abstract class members.
- **Sub classing is required:** To use an abstract class, another class must extend it.

Example of an Abstract Class:

```
public abstract class Shape {  
    String color;  
  
    public Shape(String color) {
```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

```
        this.color = color;  
    }
```

```
    // Concrete method  
    public void displayColor() {  
        System.out.println("Color: " + color);  
    }
```

```

}

// Abstract method (no body)
public abstract double calculateArea();
}

```

Abstract Method:

An abstract method is a method declared within an abstract class using the abstract keyword, but without an implementation (no method body, ends with a semicolon).

- **No body:** It defines the signature of a method but not its behavior.
- **Must be in an abstract class:** Abstract methods can only exist within abstract classes (or interfaces).
- **Subclass responsibility:** Any concrete subclass extending an abstract class must provide an implementation for all inherited abstract methods, or else the subclass itself must also be declared abstract.
- **Cannot be final, static, or private:** These modifiers are incompatible with the concept of requiring subclass implementation.

Example of an Abstract Method Implementation:

```

public class Circle extends Shape {
    double radius;

    public Circle(String color, double radius) {
        super(color);
        this.radius = radius;
    }

    @Override
    public double calculateArea() {
        return Math.PI * radius * radius;
    }
}

```

Purpose:

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

Abstract classes and methods enforce a contract for subclasses, ensuring that certain methods are implemented while allowing for common functionality to be shared. This promotes code reusability, maintainability, and a clear hierarchical structure in your application.

Examples of Java Abstract Class

1. Example of Abstract Class that has Abstract method

Below is the implementation of the above topic:

```
// Abstract class
abstract class Sunstar {
    abstract void printInfo();
}

// Abstraction performed using extends
class Employee extends Sunstar {
    void printInfo()
    {
        String name = "avinash";
        int age = 21;
        float salary = 222.2F;
        System.out.println(name);
        System.out.println(age);
        System.out.println(salary);
    }
}

// Base class
class Base {
    public static void main(String args[])
    {
        Sunstar s = new Employee();
        s.printInfo();
    }
}
```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

Output

avinash

2. Abstract Class having constructor, data member, and methods

Elements abstract class can have

- data member
- abstract method
- method body (non-abstract method)
- constructor
- main() method.

Below is the implementation of the above topic:

// Java Program to implement Abstract Class having constructor, data member, and methods

```
import java.io.*;
```

```
abstract class Subject {
    Subject() {
        System.out.println("Learning Subject");
    }
    abstract void syllabus();
    void Learn(){
        System.out.println("Preparing Right Now!");
    }
}

class IT extends Subject {
    void syllabus(){
        System.out.println("C , Java , C++");
    } }

class GFG {
    public static void main(String[] args) {
```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY

UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

```
Subject x=new IT();
```

```
x.syllabus();  
x.Learn();  
}  
}
```

Output

Learning Subject

C , Java , C++

Preparing Right Now!

3.5 INTERFACES AND MULTIPLE INHERITANCE THROUGH INTERFACES

Java Interfaces:

An interface in Java is a blueprint of a class. It defines a set of abstract methods (methods without an implementation) that a class must implement if it declares to implement that interface. Interfaces can also contain default methods (with implementation since Java 8), static methods, and constant variables. They establish a contract for behavior, ensuring that any class implementing an interface will provide the specified methods.

Multiple Inheritance through Interfaces:

While a Java class can only extend one superclass, it can implement multiple interfaces. This mechanism allows a class to achieve a form of multiple inheritance by inheriting behavior contracts from various interfaces.

Defining Interfaces: You define multiple interfaces, each specifying a particular set of behaviors.

```
interface Flyable {  
    void fly();  
}  
  
interface Swimmable {  
    void swim();  
}
```

Implementing Multiple Interfaces: A class can then implement these multiple interfaces, thereby committing to provide implementations for all the abstract methods declared in those interfaces.

```
class Duck implements Flyable, Swimmable {
    @Override
    public void fly() {
        System.out.println("Duck is flying.");
    }

    @Override
    public void swim() {
        System.out.println("Duck is swimming.");
    }
}
```

Let's consider the example of Vehicles like bicycles, cars and bikes share common functionalities, which can be defined in an interface, allowing each class (e.g., Bicycle, Car, Bike) to implement them in its own way. This approach ensures code reusability, scalability and consistency across different vehicle types.

Example: This example demonstrates how an interface can be used to define common behavior for different classes (Bicycle and Bike) that implement the interface.

```
import java.io.*;

interface Vehicle {
    // Abstract methods defined
    void changeGear(int a);
    void speedUp(int a);
    void applyBrakes(int a);
}

// Class implementing vehicle interface
class Bicycle implements Vehicle{
    int speed;
    int gear;
    // Change gear
    @Override
```

```
public void changeGear(int newGear){
    gear = newGear;
}
// Increase speed
@Override
public void speedUp(int increment){
    speed = speed + increment;
}
// Decrease speed
@Override
public void applyBrakes(int decrement){
    speed = speed - decrement;
}
public void printStates() {
    System.out.println("speed: " + speed + " gear: " + gear); }
}
// Class implementing vehicle interface
class Bike implements Vehicle {
    int speed;
    int gear;
    // Change gear
    @Override
    public void changeGear(int newGear){
        gear = newGear;
    }
    // Increase speed
    @Override
```

```
public void speedUp(int increment){
    speed = speed + increment;
}
// Decrease speed
@Override
public void applyBrakes(int decrement){
    speed = speed - decrement;
}
public void printStates() {
    System.out.println("speed: " + speed + " gear: " + gear); }
}
class Main
{
    public static void main (String[] args)
    {
        // Instance of Bicycle(Object)
        Bicycle bicycle = new Bicycle();
        bicycle.changeGear(2);
        bicycle.speedUp(3);
        bicycle.applyBrakes(1);
        System.out.print("Bicycle present state : ");
        bicycle.printStates();
        // Instance of Bike (Object)
        Bike bike = new Bike();
        bike.changeGear(1);
        bike.speedUp(4);
        bike.applyBrakes(3);
```

```
System.out.print("Bike present state : ");  
bike.printStates();  
}  
}
```

3.6 INTRODUCTION TO PACKAGES AND ACCESS CONTROL

In Java, packages and access controls are fundamental concepts for organizing code and managing the visibility and accessibility of classes, interfaces, and their members.

Packages:

A package is a mechanism to encapsulate a group of related classes, interfaces, and sub packages into a logical namespace. They help in organizing code, preventing naming conflicts, and providing controlled access.

Purpose:

- **Organization:** Grouping related types makes large projects more manageable.
- **Namespace Management:** Allows classes with the same name to exist in different packages without conflict (e.g., com.example.app.User and com.example.admin.User).
- **Access Control:** Packages play a crucial role in defining the scope of access modifiers.

Declaration:

A package is declared at the beginning of a Java source file using the package keyword:

```
package com.example.myapp;  
  
public class MyClass {  
    // ...  
}
```

Usage: To use classes from another package, you use the import statement:

```
import java.util.ArrayList; // Imports a specific class  
  
import com.example.myapp.*; // Imports all classes from a package
```

The java.lang package is automatically imported in every Java program.

2. Access Controls (Access Modifiers):

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY

UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

Access modifiers in Java define the visibility and accessibility of classes, interfaces, methods, variables, and constructors. There are four main access modifiers:

(i) **private:**

- **Scope:** Accessible only within the class where it is declared.
- **Use Case:** Encapsulating internal implementation details, preventing direct external modification.

```
class MyClass {  
    private int privateVar; // Accessible only within MyClass  
}
```

(ii) **default (Package-Private):**

- **Scope:** Accessible only within the same package. No keyword is used; it's the default if no modifier is specified.
- **Use Case:** When classes or members are intended for use by other components within the same package but not outside.

```
class MyClass {  
    int defaultVar; // Accessible within the same package  
}
```

(iii) **protected:**

- **Scope:** Accessible within the same package and by subclasses (even if the subclass is in a different package).
- **Use Case:** Allowing controlled access to members for inheritance purposes.

```
class Parent {  
    protected int protectedVar; // Accessible by subclasses and within the same package }  
}
```

(iv) **public:**

- **Scope:** Accessible from any class in any package.

- **Use Case:** Exposing functionality intended for general use throughout the application.

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY

UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

```
public class MyClass {  
    public void publicMethod() { // Accessible from anywhere  
    // ...  
}  
}
```

Types of Java Packages

- Built-in Packages
- User-defined Packages

1. Built-in Packages

These packages consist of a large number of classes which are a part of Java **API**. Some of the commonly used built-in packages are:

- **java.lang**: Contains language support classes(e.g classes which defines primitive data types, math operations). This package is automatically imported.
- **java.io**: Contains classes for supporting input / output operations.
- **java.util**: Contains utility classes which implement data structures like Linked List, Dictionary and support ; for Date / Time operations.
- **java.applet**: Contains classes for creating Applets.
- **java.awt**: Contain classes for implementing the components for graphical user interfaces (like button , ;menus etc). 6)
- **java.net**: Contain classes for supporting networking operations.

2. User-defined Packages

These are the packages that are defined by the user.

1. Create the Package:

First we create a directory myPackage (name should be same as the name of the package). Then

create the MyClass inside the directory with the first statement being the package names.

Example:

```
package myPackage;
```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY

UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

```
public class MyClass  
{  
  public void getNames(String s)  
  {  
    System.out.println(s);  
  }  
}
```

2. Use the Class in Program:

Now we will use the **MyClass** class in our program.

```
import myPackage.MyClass;  
public class Classnew {  
  public static void main(String args[]) {  
    // Initializing the String variable with a value  
    String s = "Welcome";  
    // Creating an instance of class MyClass in the package.  
    MyClass o = new MyClass();  
    o.getNames(s);  
  }  
}
```

Note: MyClass.java must be saved inside the myPackage directory since it is a part of the package.

Static Import in Java

Static Import in Java is about simplifying access to static members and separates it from the broader discussion of user-defined packages.

Static import is a feature introduced in Java programming language (versions 5 and above) that allows members (fields and methods) defined in a class as public **static** to be used in Java code without specifying the class in which the field is defined.

Example:

```
import static java.lang.System.*;
```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY

UNIT III | 24CS302-OBJECT ORIENTED PROGRAMMING USING JAVA

```
class Class1 {  
    public static void main(String args[]) {  
        // We don't need to use 'System.out' as imported using static.  
        out.println("Welcome");  
    }  
}
```

Output

Welcome

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY